Web Engineering & Development (SWE 363)

Back-End Development: Node.js & Express

In today's lecture:

- Serving static files
- Routes: GET and POST
- Middleware and `next()`
- Third-party middleware (morgan)
- The Express application generator

Reference:

- Zybook: 6.4

Activity: Warm-up, remember Lecture 15? (2 min)

```
const express = require("express");
const app = _______();

app.___("/hello", function(req, res) {
  res.send("<h1>Hello, Express!</h1>");
});

app._______(3000, function() {
  console.log("Listening on port 3000...");
});
```

1. Fill in the three blanks. Word bank: `listen` , `get` , `express` , `send`
2. True or False: `express()` starts the Express server.
3. Which method sends an object as JSON: `res.send()` or `res.json()` ?
4. What does the browser get for `http://localhost:3000/bye` ?

6.4 Express

The public Directory

```
myproject
├── node_modules
│   └── express
├── public
│   └── hello.html
├── package.json
└── server.js
```

- `node_modules/express` → the installed **express** package
- **public** → stores **static files** (HTML, images, etc.) served by the Express server
- **server.js** → the application code, run with `node server.js`

Serving Static Files

```
// server.js
const express = require("express");
const app = express();

// Serve static files from the public dir
app.use(express.static("public"));

// Start the web server
app.listen(3000, function() {
  console.log("Listening on port 3000...");
});
```

- `express.static()` → names the directory with the static files
- `app.use()` → applies it to the Express app
- `http://localhost:3000/hello.html` → the server responds with `hello.html`

Activity: Predict what the browser shows

```
myproject
├── public
│   └── hello.html
└── server.js        // app.use(express.static("public"));
```

1. `http://localhost:3000/hello.html`
2. `http://localhost:3000/bye.html`
3. `http://localhost:3000/public/hello.html`
4. To serve `images/dog.jpg`, the `images` directory must be in `public` or `node_modules`?

Routes

> An Express **route** = a URL **path** + an HTTP request **method**, with an assigned **callback function**

```
app.method(path, callback)
```

- `app` → the Express instance
- `method` → the HTTP request method (`get` , `post` , etc.)
- `path` → the URL path
- `callback` → the function executed when the route **matches**
  - Parameters `req` and `res` → the HTTP request and response

GET vs. POST Routes

```
// Called for GET request to http://localhost:3000/hello
app.get("/hello", function(req, res) {
  res.send("<h1>Hello, Express!</h1>");
});

// Called for POST request to http://localhost:3000/goodbye
app.post("/goodbye", function(req, res) {
  res.send("<h1>Goodbye, Express!</h1>");
});
```

- **GET** → requests a resource (the most common browser request)
- **POST** → often sends information from a web form: `<form method="POST">`
- A route matches only when **both** the method and the path match

Activity: Which route matches? (server on port 3000)

```
app.get("/hello", callback);
app.post("/goodbye", callback);
app.get("/droids", callback);
```

1. A POST request to `/hello`: `/hello`, `/goodbye`, or neither?
2. Which URL matches `/droids`?

   `http://localhost/droids`, `http://localhost:3000/droidssss`, or `http://localhost:3000/Droids`
3. Which route matches a POST request to `http://localhost:3000/apply`?

   `app.get("/apply", cb)`, `app.post("/apply", cb)`, or `app.post("http://localhost:3000/apply", cb)`

What is Middleware?

> A **middleware function** (or just **middleware**) examines or modifies the **request** and/or **response** objects

Why use middleware?

- Runs **before** the route, for **every** request
- Write common code **once** with `app.use()`, instead of in every route
- Ex: `express.static()` serves files, **morgan** logs requests

Three parameters: `req` (request), `res` (response), `next` (call when done)

```
Request → middleware → next() → route → Response
```

A Middleware That Logs Requests

```
const logRequest = function(req, res, next) {
  console.log(`Request: ${req.method} for ${req.path}`);
  next();
};

app.use(logRequest);
app.get("/hello", function(req, res) {
  res.send("<h1>Hello, Express!</h1>");
});
// app.listen(3000, ...) as before
```

Console after the browser requests `http://localhost:3000/hello` :

```
Listening on port 3000...
Request: GET for /hello
```

Activity: What happens if (each change on its own)

```
const logRequest = function(req, res, next) {
  console.log(`Request: ${req.method} for ${req.path}`);
  next();
};
app.use(logRequest);
app.get("/hello", function(req, res) {
  res.send("<h1>Hello, Express!</h1>");
});
```

1. The browser requests `/`, which has no route: what is logged?
2. `app.use()` is removed: what does GET `/hello` log?
3. `next()` is removed: what does GET `/hello` log? What does the browser show?
4. `app.use()` is moved after `app.get()`: what does GET `/hello` log?

Middleware: Order and `next()`

- `next()` → allows **other middleware** and the **route** to execute
- No `next()` → the request stops in the middleware
- `app.use()` **before** the routes → the middleware runs first

```
app.use(logRequest);      →  logs "Request: GET for /hello", then /hello responds
app.get("/hello", ...);

app.get("/hello", ...);   →  /hello responds; logRequest() never runs
app.use(logRequest);
```

- `express.static("public")` is also enabled with `app.use()`

Third-Party Middleware: morgan

Many useful middleware functions can be **installed with npm**

```
npm install morgan
```

```
const express = require("express");
const morgan = require("morgan");
const app = express();

// Show HTTP requests in the console
app.use(morgan("dev"));

app.listen(3000);
```

Example console logging of requests:

```
GET /hello.html 200 8.645 ms - 132
POST /hello 200 4.280 ms - 25
GET /blah 404 1.297 ms - -
```

- **morgan** logs HTTP requests
- `"dev"` → the output format

Activity: Put the steps in order

A. `app.use(morgan("dev"));` enables morgan

B. `npm install morgan`

C. The browser requests `http://localhost:3000/hello.html`

D. `const morgan = require("morgan");` imports morgan

E. The console shows `GET /hello.html 200 8.645 ms - 132`

F. `node server.js` starts the server

Popular Third-Party Middleware

Middleware	Description
morgan	Logs HTTP request information
cookie-parser	Parses the cookie header in an HTTP request
errorhandler	Aids debugging during development
csurf	Protects against cross-site request forgery (CSRF)
compression	Compresses response bodies
express-session	Manages session data on the server

Activity: Which middleware?

Choose from: morgan, cookie-parser, errorhandler, csurf, compression, express-session

1. Indicates that an HTTP request returned a 404 status code
2. Protects an Express server from a malicious CSRF attack
3. Tracks the user's temporary session data
4. Makes response bodies smaller
5. Reads the cookie header of a request
6. Aids debugging during development

Express Application Generator

> **express-generator** automatically creates an Express **application skeleton**

- Saves time writing code that is common to many Express applications
- Generated projects use the **Pug** template engine to create views (webpages)
- The views interact with the **MongoDB** database

Activity: Find the 3 bugs (exit ticket)

Goal: `http://localhost:3000/hello` shows "Hello!" and every request is logged.

```
const express = require("express");
const app = express();

const logRequest = function(req, res) {
  console.log(`Request: ${req.method} for ${req.path}`);
};

app.use(morgan("dev"));
app.use(logRequest);

app.post("/hello", function(req, res) {
  res.send("<h1>Hello!</h1>");
});

app.listen(3000);
```

Key Takeaways

- `app.use(express.static("public"))` → serves the static files in **public**
- **Route:** `app.method(path, callback)` → method **and** path must match
- **Middleware** `(req, res, next)` is enabled with `app.use()`
- Call `next()` so other middleware and the route can execute
- **Third-party middleware** is installed with npm, e.g., `morgan("dev")`
- **express-generator** creates an Express application skeleton

30m

Demo

6.3 Express

- Routes that send text and JSON
- Logging requests with morgan
- Connect the React front end

Next Class

- Express request data (6.5)
- Query strings, form data, and route parameters